Thynk Unlimited

Vehicle Insurance Management System





Presented By: **Hazra Shaikh**

SQL Project
Data Analysis

Project Overview

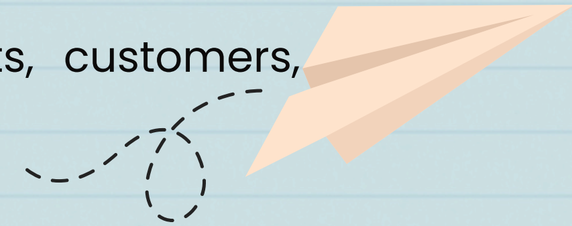
A vehicle insurance company sells insurance policies for two-wheelers, cars, SUVs and commercial vehicles
through its agents. Customers register their vehicles, buy a policy under one of the insurance plans, pay the
premium and may raise claims when an incident happens.
In this project you act as the data analyst of the company. You will create the database, load the data and answer
20 business questions using SQL. The questions start with simple filtering and grow step by step to joins,
subqueries, CTEs and window functions.

Dataset and Setup

VIMS_Dataset.xlsx - one sheet per table. Use it to explore the data or to import with the MySQL Workbench Table Data Import Wizard.

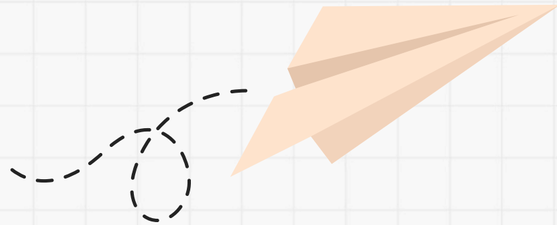
Loading order

Parent tables must be loaded before child tables: insurance_plans, agents, customers, vehicles, policies,
then payments and claims. The setup script already follows this order.



Project Requirements and Database Design

Project	Vehicle Insurance Management System (VIMS)
Database	`vims` (MySQL)
Tables	7 tables, 501 rows in total
Questions	20 analysis requirements (7 Beginner, 7 Intermediate, 6 Advanced)
Reference date	2026-10-20
Files	VIMS_Dataset.xlsx (data), vims_setup.sql (create tables + load data), this document, VIMS Master Answers.docx



Entity Relationship Diagram



Business Rules and Data Notes

- All amounts are in Indian Rupees (Rs.) and all dates are in YYYY-MM-DD format.
- The reference date of the project is 2026-10-20. Use this fixed date in your queries instead of CURDATE() so that your output matches the answer file.
- IDV (Insured Declared Value) is between 92% and 100% of the vehicle value. Premium = IDV x base_rate_pct / 100.
- Every policy runs for one year. A renewal is stored as a new policy row for the same vehicle. Policy status is Active, Expired or Cancelled.
- A policy is paid either in one payment or in two instalments. Cancelled policies and the four newest policies have no payment rows.
- Claim status is Settled (paid, has a settlement_date), Approved (approved but not yet paid), Under Review or Rejected (approved_amount = 0).
- Some data is deliberately incomplete to practise NULL handling and outer joins: 6 customers have no email, 6 customers own no vehicle, 10 vehicles never had a policy.

Project Questions

Q1. Display the full name, city and email of all customers who live in Pune or Mumbai and use a Gmail address.

```
SELECT CONCAT(first_name, " ",last_name) AS Full_Name, city, email
FROM customers
WHERE city IN ('Pune', 'Mumbai')
AND email LIKE '%@gmail.com';
```

Full_Name	city	email
Priya Khan	Mumbai	priya.khan@gmail.com
Rohan Gupta	Pune	rohan.gupta@gmail.com
Vikas Kale	Pune	vikas.kale@gmail.com
Divya Mehta	Pune	divya.mehta@gmail.com
Harsh Bhosale	Mumbai	harsh.bhosale@gmail.com
Divya Khan	Pune	divya.khan@gmail.com



Project Questions

Q2. List the registration number, make, model and vehicle value of the 5 most expensive vehicles.

```
SELECT registration_no , make, model, vehicle_value
FROM vehicles
ORDER BY vehicle_value DESC
LIMIT 5;
```

registration_no	make	model	vehicle_value
KA01KZ2503	Mahindra	XUV700	1848000.00
GJ05YW9290	Tata	Harrier	1840000.00
GJ01BU6767	Hyundai	Creta	1550000.00
MP09XE1013	Kia	Seltos	1518000.00
MH31FW9234	Hyundai	Creta	1426000.00



Project Questions

Q3. Find the number of vehicles and the average vehicle value for each vehicle type.

```
SELECT vehicle_type,
COUNT(vehicle_id) AS total_vehicles ,
ROUND(AVG(vehicle_value),2) AS Avg_Value
FROM vehicles
GROUP BY vehicle_type;
```

	vehide_type	total_vehides	Avg_Value
▶	Car	42	634571.43
	Two-Wheeler	37	56675.68
	Commercial	9	393000.00
	SUV	12	1052333.33



Project Questions

Q4. Show all policies whose premium is between Rs. 30,000 and Rs. 60,000 (inclusive), highest premium first.

```
SELECT policy_id, vehicle_id, plan_id,premium_amount, policy_status
FROM policies
WHERE premium_amount BETWEEN 30000 and 60000
ORDER BY premium_amount DESC;
```

policy_id	vehicle_id	plan_id	premium_amount	policy_status
POL0091	V018	P05	59035.50	Active
POL0092	V036	P05	55890.00	Active
POL0054	V100	P03	51006.00	Expired
POL0094	V030	P06	48735.00	Active
POL0035	V098	P03	39900.00	Expired
POL0040	V027	P05	39244.50	Expired
POL0003	V054	P06	32400.00	Expired
POL0069	V034	P02	32292.00	Expired
POL0001	V006	P06	30360.00	Expired
NULL	NULL	NULL	NULL	NULL

Project Questions

Q5. Find the customers whose email address is not recorded. Show customer id, name and city.

```
SELECT customer_id, CONCAT(first_name, " ",last_name) AS customer_name, city
FROM customers
WHERE  email IS NULL
OR TRIM(email) = '';
```

customer_id	customer_name	city
C007	Nidhi Khan	Bengaluru
C019	Swati Sharma	Pune
C033	Rohan Nair	Pune
C048	Isha Khan	Pune
C062	Aditi Kulkarni	Nagpur
C075	Meera Bhosale	Delhi



Project Questions

Q6. List all Petrol vehicles of Maruti Suzuki, Hyundai or Tata that were manufactured in 2020 or later, with model and manufacture year.

```
SELECT vehicle_id , make, model, manufacture_year, fuel_type
FROM vehicles
WHERE make IN ("Maruti Suzuki", "Hyundai", "Tata")
AND fuel_type = "Petrol"
AND manufacture_year >= 2020;
```

vehicle_id	make	model	manufacture_year	fuel_type
V002	Maruti Suzuki	Baleno	2020	Petrol
V006	Tata	Nexon	2021	Petrol
V009	Maruti Suzuki	Baleno	2023	Petrol
V043	Maruti Suzuki	Baleno	2025	Petrol
V046	Tata	Nexon	2021	Petrol
V052	Tata	Nexon	2024	Petrol
V059	Tata	Nexon	2023	Petrol
V089	Maruti Suzuki	Swift	2022	Petrol
NULL	NULL	NULL	NULL	NULL

Project Questions

Q7. As of 2026-10-20, list all Active policies that will expire within the next 90 days, with the number of days left.

```
SELECT policy_id, vehicle_id,
end_date,
DATEDIFF(end_date, '2026-10-20')
AS days_left
FROM policies
WHERE policy_status = 'Active'
AND end_date BETWEEN '2026-10-20'
AND DATE_ADD('2026-10-20',
INTERVAL 90 DAY)
ORDER BY end_date;
```

policy_id	vehide_id	end_date	days_left
POL0083	V055	2026-11-06	17
POL0084	V082	2026-11-06	17
POL0085	V009	2026-11-19	30
POL0086	V022	2026-11-24	35
POL0087	V086	2026-12-06	47
POL0088	V067	2027-01-02	74
POL0089	V008	2027-01-05	77
POL0090	V023	2027-01-14	86

Project Questions

Q8. Show each customer's name together with the registration number, make and model of the vehicles they own. Sort by customer name and show only the first 15 rows.

```
SELECT CONCAT(first_name, " ",last_name) AS customer_name , v.registration_no, v.make, v.model
FROM vehicles v
JOIN customers c
ON v.customer_id = c.customer_id
LIMIT 15;
```

customer_name	registration_no	make	model
Swati Reddy	MH02FB4580	Tata	Nexon EV
Vihaan Nair	GJ05UY1582	Maruti Suzuki	Baleno
Sakshi Rao	MP09XB4113	Ather	450X
Omkar Joshi	MP09CF2073	Toyota	Glanza
Rohan Gupta	MH12WX1714	Hero	Splendor Plus
Nidhi Khan	KA01FU9465	Tata	Nexon
Ananya Pawar	GJ01BU6767	Hyundai	Creta
Yash Menon	MH12HB1247	TVS	Jupiter
Nidhi Menon	TS09WF3811	Maruti Suzuki	Baleno
Priya Khan	MH02LN5569	Royal Enfield	Classic 350
Nikhil Pawar	MH12CA6641	Ashok Leyland	Dost
Shruti Iyer	MH31TH8771	Honda	Activa 6G
Aditya Mehta	KA09SU2890	Mahindra	Scorpio N
Aditi Patil	MP09XP6547	TVS	Jupiter
Aarav Chavan	MH02VA5133	Tata	Nexon EV

Project Questions

Q9. Prepare a policy report showing policy id, customer name, vehicle registration number, plan name, agent name and premium for all Active policies. Show only the top 10 by premium.

```
SELECT p.policy_id,
CONCAT(c.first_name,"",c.last_name)AS
customer_name ,
v.registration_no, i.plan_name,
a.agent_name, p.premium_amount
FROM vehicles v
JOIN customers c
ON c.customer_id = v.customer_id
JOIN policies p
ON p.vehicle_id = v.vehicle_id
JOIN insurance_plans i
ON p.plan_id = i.plan_id
JOIN agents a
ON p.agent_id = a.agent_id
WHERE p.policy_status = 'Active'
ORDER BY p.premium_amount DESC
LIMIT 10;
```

policy_id	customer_name	registration_no	plan_name	agent_name	premium_amount
POL0093	Ananya Pawar	GJ01BU6767	Zero Depreciation Plus	Swati Kale	77500.00
POL0091	Aditi Kulkarni	MH31FW9234	Comprehensive Platinum	Neha Joshi	59035.50
POL0092	Neha Gupta	MH02FP9405	Comprehensive Platinum	Sneha Patil	55890.00
POL0094	Rutuja Menon	MH31BW6031	Zero Depreciation Plus	Neha Joshi	48735.00
POL0103	Aditi Menon	RJ14PE2493	Comprehensive Gold	Manish Gupta	28317.60
POL0085	Nidhi Menon	TS09WF3811	Comprehensive Gold	Arjun Menon	25422.00
POL0102	Ajay Khan	KA01HP2785	Comprehensive Gold	Rohan Kulkarni	23826.00
POL0105	Vikas Das	GJ01GC7496	Comprehensive Gold	Amit Deshmukh	20767.00
POL0106	Rohan Iyer	GJ01PH9763	Comprehensive Gold	Rahul Bhosale	20208.40
POL0104	Vihaan Nair	GJ05UY1582	Comprehensive Gold	Priya Nair	17404.00

Project Questions

Q10. Calculate the total premium collected (successful payments only) and the number of payments for each payment mode.

```
SELECT payment_mode,
COUNT(payment_id)AS No_of_payments,
SUM(amount_paid)AS Total_amount_collected
FROM payments
WHERE payment_status = 'Success'
GROUP BY payment_mode;
```

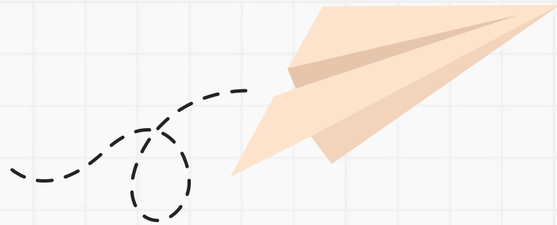
payment_mode	No_of_payments	Total_amount_collected
UPI	45	585812.20
Net Banking	18	241113.50
Debit Card	18	150468.40
Credit Card	23	274291.80
Cheque	3	24268.60
Cash	5	36372.80

Project Questions

Q11. For each incident type show the number of claims, total claimed amount and total approved amount.
Show only incident types having at least 3 claims.

```
SELECT incident_type,
COUNT(claim_id) AS total_claims,
SUM(claim_amount)AS  total_claimed,
SUM(approved_amount)AS total_approved
FROM claims
GROUP BY incident_type
HAVING COUNT(claim_id) >= 3;
```

incident_type	total_claims	total_claimed	total_approved
Accident	50	3795900.00	2190000.00
Vandalism	3	416100.00	321900.00
Windshield Damage	6	102000.00	43600.00
Theft	4	2552800.00	334600.00
Natural Calamity	4	688100.00	88200.00

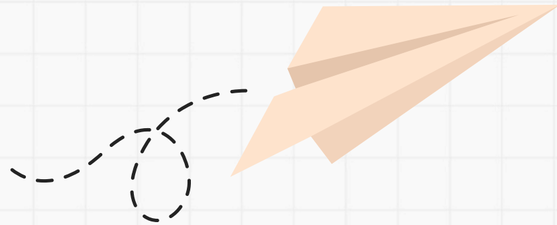


Project Questions

Q12. Find the customers who have not registered any vehicle yet.

```
SELECT c.customer_id,
CONCAT(c.first_name, " ",c.last_name) AS customer_name , c.city
FROM customers c
LEFT JOIN vehicles v
ON c.customer_id = v.customer_id
WHERE registration_no IS NULL;
```

customer_id	customer_name	city
C004	Tanvi Bhosale	Delhi
C019	Swati Sharma	Pune
C035	Ajay More	Nashik
C037	Aditi Singh	Hyderabad
C047	Kavya Iyer	Surat
C071	Amit Deshmukh	Ahmedabad



Project Questions

Q13. Find the vehicles that have never had a policy. Show registration number, make, model and owner name.

```
SELECT v.vehicle_id,
v.registration_no, v.make,
v.model,
CONCAT(c.first_name, "
",c.last_name) AS Owner_name
FROM vehicles v
LEFT JOIN customers c
ON c.customer_id =
v.customer_id
LEFT JOIN policies p
ON p.vehicle_id = v.vehicle_id
WHERE policy_id IS NULL;
```

vehicle_id	registration_no	make	model	Owner_name
V005	MH12WX1714	Hero	Splendor Plus	Rohan Gupta
V038	MH31MX9452	Toyota	Glanza	Rohan Joshi
V042	MH15MH3447	Maruti Suzuki	Baleno	Swati More
V051	GJ05BU5124	TVS	Jupiter	Neha Khan
V053	DL3CJK2807	Royal Enfield	Classic 350	Mayur Deshmukh
V059	MH12UB2903	Tata	Nexon	Divya Mehta
V068	MP09XE1013	Kia	Seltos	Aditi Patil
V073	MH31LJ2520	Honda	Activa 6G	Aditi Shah
V085	MP09MY9065	Hyundai	Creta	Omkar Joshi
V087	DL3CJU9598	Mahindra	Scorpio N	Tanvi Verma

Project Questions

Q14. For every agent show the number of policies sold, total premium and the commission earned (premium x commission %). Agents with no sales must also appear.

```
SELECT a.agent_id, a.agent_name, a.city,
COUNT(p.policy_id) AS policies_sold,
COALESCE(SUM(p.premium_amount), 0) AS
total_premium,
COALESCE(SUM(p.premium_amount *
a.commission_pct / 100), 0) AS
commission_earned
FROM agents a
LEFT JOIN policies p
ON a.agent_id = p.agent_id
GROUP BY a.agent_id, a.agent_name, a.city;
```

agent_id	agent_name	city	policies_sold	total_premium	commission_earned
A001	Rohan Kulkarni	Pune	18	248103.90	24810.3900000
A002	Sneha Patil	Pune	11	107885.80	5394.2900000
A003	Amit Deshmukh	Mumbai	6	93815.00	9381.5000000
A004	Priya Nair	Bengaluru	6	66732.80	6673.2800000
A005	Vikram Singh	Delhi	8	98501.30	7880.1040000
A006	Neha Joshi	Pune	8	246019.50	24601.9500000
A007	Sandeep Rao	Hyderabad	3	22692.60	1134.6300000
A008	Pooja Mehta	Ahmedabad	6	50382.20	2519.1100000
A009	Karan Malhotra	Mumbai	9	108958.10	5447.9050000
A010	Anjali Iyer	Chennai	6	120675.00	9654.0000000
A011	Rahul Bhosale	Nashik	7	83611.00	5016.6600000
A012	Divya Reddy	Hyderabad	5	65178.60	4888.3950000
A013	Manish Gupta	Delhi	8	130890.40	10471.2320000
A014	Swati Kale	Nagpur	2	78146.00	3907.3000000
A015	Arjun Menon	Bengaluru	7	63649.80	3182.4900000

Project Questions

Q15. For each insurance plan find the total claims, approved claims (Settled or Approved), rejected claims and the approval percentage.

```
SELECT i.plan_name,
COUNT(c.claim_id) AS total_claims,
SUM(CASE WHEN c.claim_status IN ('Settled', 'Approved') THEN 1 ELSE 0 END) AS approved_claims,
SUM(CASE WHEN c.claim_status = 'Rejected' THEN 1 ELSE 0 END) AS rejected_claims,
ROUND(SUM(CASE WHEN c.claim_status IN ('Settled', 'Approved') THEN 1 ELSE 0 END) * 100.0
/ COUNT(c.claim_id), 2) AS approval_pct
FROM insurance_plans i
JOIN policies p ON i.plan_id = p.plan_id
JOIN claims c ON p.policy_id = c.policy_id
GROUP BY i.plan_id, i.plan_name;
```

plan_name	total_claims	approved_claims	rejected_claims	approval_pct
Third Party Basic	15	9	3	60.00
Comprehensive Gold	20	15	3	75.00
Comprehensive Silver	17	6	5	35.29
Own Damage Saver	5	2	2	40.00
Zero Depreciation Plus	6	6	0	100.00
Comprehensive Platinum	7	3	2	42.86

Project Questions

Q16. Find customers who own more than one vehicle. Show the number of vehicles and the total value of their vehicles, and display only the top 10 customers (most vehicles first).

```
SELECT c.customer_id,
CONCAT(c.first_name, " ",c.last_name) AS
customer_name,
COUNT(v.vehicle_id) AS Vehicle_owned,
SUM(v.vehicle_value) AS Total_vehicle_value
FROM vehicles v
JOIN customers c
ON c.customer_id = v.customer_id
GROUP BY c.customer_id
HAVING COUNT(v.vehicle_id) > 1
ORDER BY Vehicle_owned DESC
LIMIT 10;
```

customer_id	customer_name	Vehicle_owned	Total_vehicle_value
C007	Nidhi Khan	3	3086000.00
C029	Nidhi Menon	3	1719000.00
C066	Aditi Patil	3	1591000.00
C039	Rohan Iyer	3	944000.00
C009	Swati Reddy	2	1004000.00
C045	Omkar Joshi	2	850000.00
C030	Rohan Gupta	2	520000.00
C040	Ananya Pawar	2	2036000.00
C012	Priya Khan	2	909000.00
C061	Anjali Menon	2	755000.00

Project Questions

Q17. List the policies whose premium is higher than the average premium of their own plan. Show the plan average next to each policy, sorted by plan and premium (highest first), and display only the first 12 rows.

```
SELECT p.policy_id, p.plan_id, p.premium_amount,
 (
 SELECT AVG(p1.premium_amount)
 FROM policies p1
 WHERE p1.plan_id = p.plan_id
 )AS plan_avg_premium
FROM policies p
WHERE p.premium_amount > (
   SELECT AVG(p1.premium_amount)
   FROM policies p1
   WHERE p1.plan_id = p.plan_id
)
ORDER BY p.plan_id, p.premium_amount DESC
LIMIT 12;
```

policy_id	plan_id	premium_amount	plan_avg_premium
POL0033	P01	11560.00	1794.827586
POL0071	P01	8702.00	1794.827586
POL0010	P01	4040.00	1794.827586
POL0037	P01	2340.00	1794.827586
POL0061	P01	2318.00	1794.827586
POL0024	P01	2152.00	1794.827586
POL0069	P02	32292.00	10969.636364
POL0014	P02	20592.00	10969.636364
POL0004	P02	15444.00	10969.636364
POL0028	P02	15028.00	10969.636364
POL0023	P02	11624.60	10969.636364
POL0054	P03	51006.00	13509.000000

Project Questions

Q18. Rank the agents inside their own city by total premium sold (highest = rank 1). Cancelled policies must be ignored.

```
WITH agent_sales AS (
  SELECT a.city, a.agent_id, a.agent_name,
 COALESCE(SUM(
       CASE
         WHEN p.policy_status != 'Cancelled'
         THEN p.premium_amount
         ELSE 0
       END
    ), 0) AS total_premium
  FROM agents a
  LEFT JOIN policies p
    ON a.agent_id = p.agent_id
  GROUP BY a.city, a.agent_id, a.agent_name
)
SELECT city, agent_name, total_premium,
  RANK() OVER ( PARTITION BY city
    ORDER BY total_premium DESC
  ) AS city_rank
FROM agent_sales
ORDER BY city, city_rank;
```

city	agent_name	total_premium	city_rank
Ahmedabad	Pooja Mehta	49688.00	1
Bengaluru	Priya Nair	66732.80	1
Bengaluru	Arjun Menon	63649.80	2
Chennai	Anjali Iyer	120675.00	1
Delhi	Manish Gupta	104556.40	1
Delhi	Vikram Singh	98501.30	2
Hyderabad	Divya Reddy	65178.60	1
Hyderabad	Sandeep Rao	22692.60	2
Mumbai	Karan Malhotra	108958.10	1
Mumbai	Amit Deshmukh	82255.00	2
Nagpur	Swati Kale	78146.00	1
Nashik	Rahul Bhosale	83611.00	1
Pune	Neha Joshi	245199.50	1
Pune	Rohan Kulkarni	232767.30	2
Pune	Sneha Patil	107885.80	3

Project Questions

Q19. Show month-wise premium booked (by policy start month) for the year 2025 along with the cumulative running total.

```
SELECT DATE_FORMAT(start_date, '%Y-%m') AS
policy_month,
COUNT(policy_id) AS policies_issued,
 SUM(premium_amount) AS monthly_premium,
 SUM(SUM(premium_amount)) OVER (
      ORDER BY DATE_FORMAT(start_date, '%Y-%m')
   ) AS running_total
FROM policies
WHERE start_date >= '2025-01-01'
  AND start_date < '2026-01-01'
GROUP BY DATE_FORMAT(start_date, '%Y-%m')
ORDER BY policy_month;
```

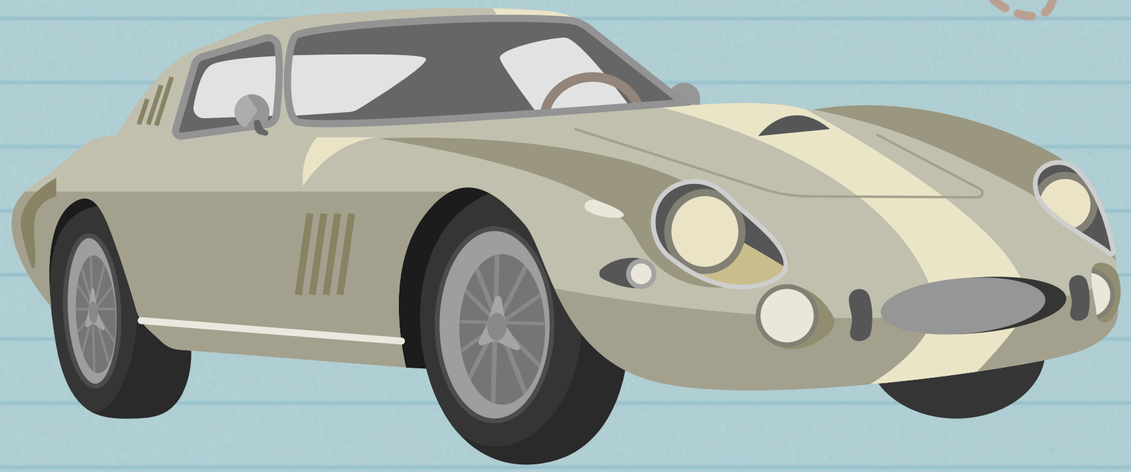
policy_month	policies_issued	monthly_premium	running_total
2025-01	6	107600.80	107600.80
2025-02	1	28980.00	136580.80
2025-03	2	19992.00	156572.80
2025-04	2	2796.00	159368.80
2025-05	1	15400.00	174768.80
2025-06	3	56426.00	231194.80
2025-07	1	8702.00	239896.80
2025-08	3	22096.00	261992.80
2025-09	4	60402.00	322394.80
2025-10	4	44475.60	366870.40
2025-11	4	55680.80	422551.20
2025-12	1	15234.20	437785.40

Project Questions

Q20. Calculate the loss ratio for every customer = (total approved claim amount / total premium of their policies) x 100. Show the 10 customers with the highest loss ratio and flag customers above 100% as 'High Risk'.

```
WITH customer_premium AS (
  SELECT  v.customer_id,
  SUM(p.premium_amount) AS total_premium
  FROM vehicles v
  JOIN policies p ON v.vehicle_id = p.vehicle_id
  GROUP BY v.customer_id),
customer_claims AS (
  SELECT  v.customer_id,
  SUM(c.approved_amount) AS total_approved
  FROM vehicles v
  JOIN policies p ON v.vehicle_id = p.vehicle_id
  JOIN claims c ON p.policy_id = c.policy_id
  GROUP BY v.customer_id)
SELECT  c.customer_id,
  CONCAT(c.first_name, ' ', c.last_name) AS customer_name,
  cp.total_premium,
  COALESCE(cc.total_approved, 0) AS total_approved,
  ROUND(
    COALESCE(cc.total_approved, 0) * 100.0 /
    NULLIF(cp.total_premium, 0), 2
  ) AS loss_ratio_pct,
  CASE
    WHEN COALESCE(cc.total_approved, 0) * 100.0 /
     NULLIF(cp.total_premium, 0) > 100  THEN 'High Risk'
    ELSE 'Normal'
  END AS risk_flag
FROM customers c
JOIN customer_premium cp ON c.customer_id = cp.customer_id
LEFT JOIN customer_claims cc ON c.customer_id = cc.customer_id
ORDER BY loss_ratio_pct DESC
LIMIT 10;
```

customer_id	customer_name	total_premium	total_approved	loss_ratio_pct	risk_flag
C014	Kunal Kulkarni	1360.00	32300.00	2375.00	High Risk
C008	Meera Sharma	1252.00	24200.00	1932.91	High Risk
C016	Yash Menon	828.00	15100.00	1823.67	High Risk
C033	Rohan Nair	1819.60	26100.00	1434.38	High Risk
C063	Riya Mehta	2970.00	41300.00	1390.57	High Risk
C022	Isha Reddy	26880.20	334600.00	1244.78	High Risk
C006	Sakshi Rao	3340.00	36700.00	1098.80	High Risk
C076	Tanvi Das	2262.00	22600.00	999.12	High Risk
C012	Priya Khan	22932.00	228800.00	997.73	High Risk
C080	Pooja Chavan	2432.00	23800.00	978.62	High Risk



Thank You Very Much

https://github.com/Hazra-Shaikh